SOUM DECO — Handle 1 Million Visits/Month for $0

The Core Principle

> **Static assets on Cloudflare Pages are unlimited and free — forever. No bandwidth cap, no request cap, no daily limit.** The entire strategy revolves around turning everything possible into a static asset.

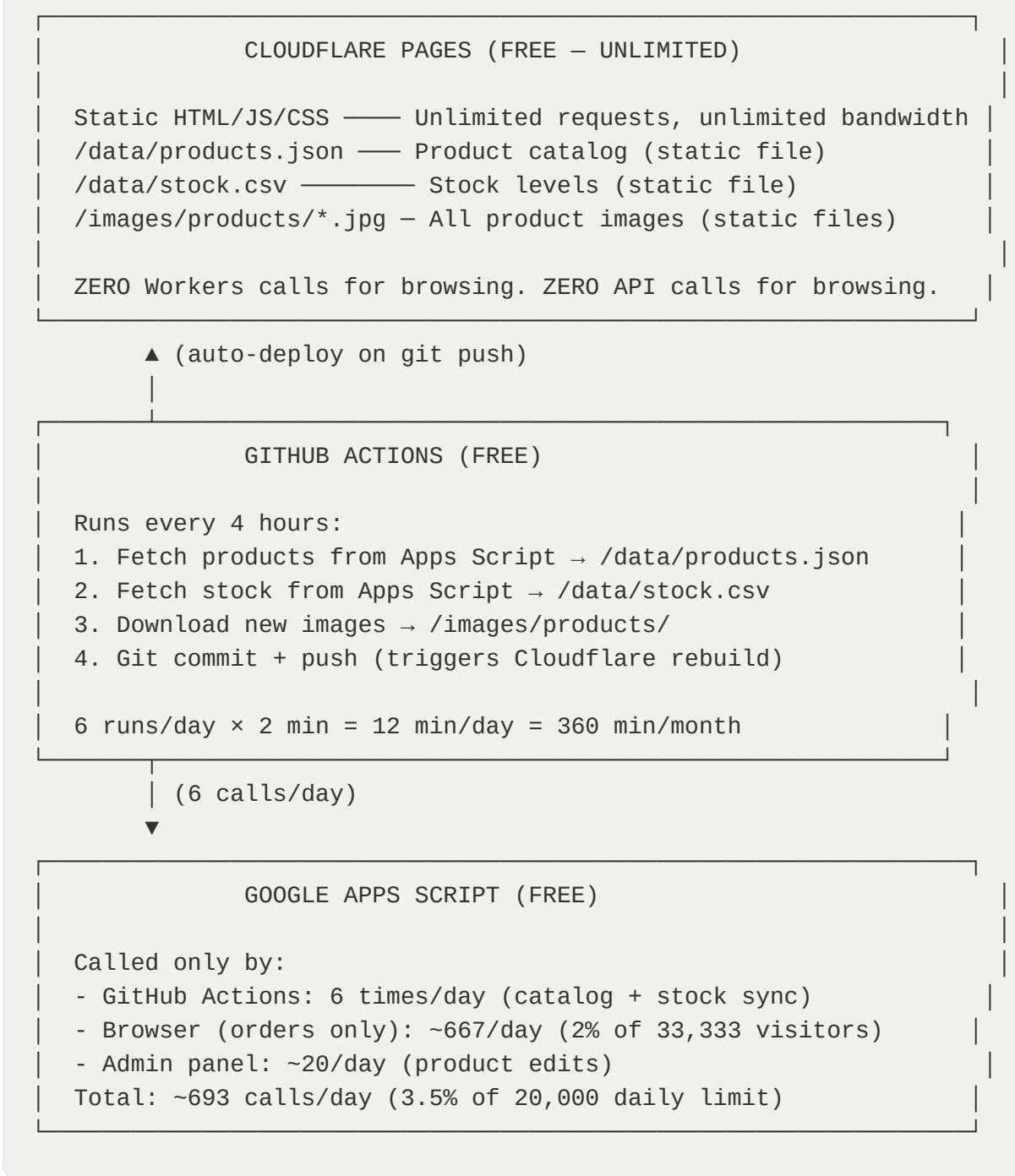
Free Tier Limits We Must Respect

Service	Free Limit	Our Usage	Headroom
Cloudflare Pages static requests	**Unlimited**	15M/month	Infinite
Cloudflare Pages bandwidth	**Unlimited**	~2 TB/month	Infinite
Cloudflare Pages files	20,000	~300	98.5% free
Cloudflare Pages builds	500/month	180/month	64% free
Cloudflare Workers/Functions	100,000/day	0/day	100% free
Cloudflare KV reads	100,000/day	0/day	100% free
Google Apps Script calls	20,000/day	~700/day	96.5% free
GitHub Actions minutes	2,000/month (private)	720/month	64% free
GitHub Actions minutes	Unlimited (public)	Any	Infinite

Every single service stays well within free tier. No risk of hitting any limit.

Architecture: "Static-First, Dynamic-Only-When-Necessary"

Plain Text



How It Works: Step by Step

For a Visitor (99.97% of traffic — ZERO dynamic calls)

1. Visitor opens `soumdeco.com`
2. Cloudflare Pages serves static HTML + JS + CSS (unlimited, free, <50ms)
3. Browser loads `/data/products.json` — a static file on Cloudflare CDN (unlimited, free, <50ms)
4. Browser loads `/data/stock.csv` — a static file on Cloudflare CDN (unlimited, free, <50ms)
5. Browser loads product images from `/images/products/` — static files (unlimited, free)
6. Visitor browses, filters, views products — all client-side, zero server calls
7. **Total server/API calls for browsing: ZERO**

For a Buyer (2% of visitors — 1 Apps Script call)

1. Visitor fills checkout form
2. Browser submits order directly to Google Apps Script (1 call)
3. Apps Script writes to Google Sheet
4. **Total API calls per order: 1**

For the Admin (rare — a few calls/day)

1. Admin logs in (client-side, no server call)
2. Admin adds/edits product → calls Apps Script (1 call per save)
3. Changes appear on site within 4 hours (next GitHub Actions sync)
4. Admin can trigger manual sync via GitHub Actions "workflow_dispatch"

Implementation Changes

Change 1: Generate Static Product JSON at Build Time

New file: `scripts/generate-static-data.py`

Python

```python
#!/usr/bin/env python3
"""
Fetches products and stock from Apps Script,
writes them as static JSON/CSV files for Cloudflare Pages.
Runs in GitHub Actions every 4 hours.
"""
import json
import urllib.request
```

```python
import os

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
REPO_ROOT = os.path.dirname(SCRIPT_DIR)
PRODUCTS_PATH = os.path.join(REPO_ROOT, "public", "data", "products.json")
STOCK_PATH = os.path.join(REPO_ROOT, "public", "data", "stock.csv")

APPS_SCRIPT_URL = "https://script.google.com/macros/s/YOUR_SCRIPT_ID/exec"

def fetch_products( ):
    url = f"{APPS_SCRIPT_URL}?action=products"
    req = urllib.request.Request(url, headers={"User-Agent": "StaticSync/1.0"})
    with urllib.request.urlopen(req, timeout=30) as resp:
        return json.loads(resp.read())

def fetch_stock():
    url = f"{APPS_SCRIPT_URL}?action=stock"
    req = urllib.request.Request(url, headers={"User-Agent": "StaticSync/1.0"})
    with urllib.request.urlopen(req, timeout=30) as resp:
        return resp.read().decode("utf-8")

def main():
    # Fetch and write products
    products = fetch_products()
    os.makedirs(os.path.dirname(PRODUCTS_PATH), exist_ok=True)
    with open(PRODUCTS_PATH, "w", encoding="utf-8") as f:
        json.dump(products, f, ensure_ascii=False)
    print(f"Wrote {len(products)} products to {PRODUCTS_PATH}")

    # Fetch and write stock
    stock_csv = fetch_stock()
    with open(STOCK_PATH, "w", encoding="utf-8") as f:
        f.write(stock_csv)
    print(f"Wrote stock data to {STOCK_PATH}")

if __name__ == "__main__":
    main()
```

Change 2: Update GitHub Actions Workflow

File: `.github/workflows/auto-sync.yml`

YAML

```yaml
name: Auto-Sync Static Data + Images
on:
  schedule:
```

```yaml
    # Every 4 hours (6 times/day = 180 builds/month, well under 500 limit)
    - cron: "0 */4 * * *"
  workflow_dispatch: # Manual trigger for immediate updates

permissions:
  contents: write

jobs:
  sync:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4
        with:
          token: ${{ secrets.GITHUB_TOKEN }}
          fetch-depth: 1

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Generate static product data
        run: python3 scripts/generate-static-data.py

      - name: Sync product images
        run: python3 scripts/auto-sync.py

      - name: Check for changes
        id: changes
        run: |
          if [ -n "$(git status --porcelain)" ]; then
            echo "changed=true" >> $GITHUB_OUTPUT
          else
            echo "changed=false" >> $GITHUB_OUTPUT
          fi

      - name: Commit and push
        if: steps.changes.outputs.changed == 'true'
        run: |
          git config user.name "Auto-Sync Bot"
          git config user.email "actions@github.com"
          git add public/data/ public/images/products/ public/image-manifest.js
          git commit -m "sync: update products + stock + images [skip ci]"
          git push
```

Important: The `[skip ci]` in the commit message prevents infinite build loops on some CI systems. On Cloudflare Pages, the push still triggers a deploy.

Change 3: Client Fetches Static JSON Instead of Apps Script

Update `src/hooks/use-catalog.ts` **:**

TypeScript

```
// BEFORE (calls Apps Script on every page load — breaks at scale):
// const fetched = await clientListProducts(); // hits Apps Script

// AFTER (loads static JSON from Cloudflare CDN — unlimited, free, <50ms):
const refresh = useCallback(async () => {
  try {
    // Show cached data instantly
    const instantCached = loadCatalog();
    if (instantCached.length > 0) {
      setProducts(instantCached.map(optimizeCloudinaryUrls).map(fixCategoryTypo
      setLoading(false);
    }

    // Fetch from STATIC JSON file (served by Cloudflare Pages CDN — free, unli
    const res = await fetch("/data/products.json", {
      cache: "no-cache", // Always get latest deployed version
    });
    if (!res.ok) return;
    const data = await res.json();
    if (!Array.isArray(data) || data.length === 0) return;

    // Process and save
    let products = data.map(normalizeProduct).map(optimizeCloudinaryUrls).map(f
    products.sort((a, b) => (a.sortOrder ?? 999) - (b.sortOrder ?? 999));
    saveCatalog(products);
    setProducts(products);
    setLoading(false);
  } catch (err) {
    console.warn("[Catalog] Static fetch failed, using cache:", err);
    setLoading(false);
  }
}, []);
```

Change 4: Stock Loads from Static CSV

Update `src/hooks/use-stock.ts` **:**

TypeScript

```
// BEFORE: fetches from Apps Script every 30 minutes
// AFTER: fetches from static CSV file (updated every 4 hours)
```

```
const fetchStock = useCallback(async () => {
  try {
    // Fetch from static CSV file on Cloudflare Pages (free, unlimited)
    const res = await fetch("/data/stock.csv", { cache: "no-cache" });
    if (!res.ok) return;
    const text = await res.text();
    const map = parseCsv(text);
    setStockMap(map);
    // Cache in localStorage
    localStorage.setItem(STOCK_CACHE_KEY, JSON.stringify({
      data: map,
      timestamp: Date.now(),
    }));
  } catch {
    // Use cached stock data
  }
}, []);
```

Change 5: Orders Still Go Direct to Apps Script (No Change Needed)

The current `clientSubmitOrder` function already submits directly to Apps Script from the browser. This is perfect because:

- Only 2% of visitors place orders (~667/day)
- Apps Script can handle 20,000 calls/day
- No Workers/Functions consumed
- No changes needed

Change 6: Images Served from Cloudflare Pages (Already Mostly Done)

The existing auto-sync already downloads images to `/public/images/products/`. The image manifest rewrites Cloudinary URLs to local paths. This is already the correct approach for unlimited free serving.

Ensure all images are local (no Cloudinary fallback needed at scale):

TypeScript

```
// In the auto-sync script, ensure ALL images are downloaded
// The current script already does this — just verify it runs in the new workfl
```

Change 7: Remove the Next.js Framework (Optional but Recommended)

Why: Next.js adds complexity (build time, Worker size, compatibility issues) but provides zero benefit when the site is a pure static SPA with no SSR, no API routes, and no image optimization.

Replace with Vite + React (same components, faster builds, smaller output):

```bash
# This is optional but eliminates the Next.js 16 + Cloudflare incompatibility e
npm install vite @vitejs/plugin-react
# Move src/app/page.tsx → src/App.tsx
# Remove next.config.ts, middleware, API routes (all dead code anyway)
```

If you keep Next.js: Use `output: 'export'` in `next.config.ts` to generate a pure static site:

```typescript
// next.config.ts
const nextConfig: NextConfig = {
  output: 'export', // Pure static HTML export — no server needed
  images: { unoptimized: true },
  typescript: { ignoreBuildErrors: true },
};
```

With `output: 'export'`, Cloudflare Pages serves it as pure static files. **Zero Workers consumed. Unlimited traffic. Free forever.**

Handling Peak Traffic (Viral Moments)

What if you get 10x traffic one day (333,000 visits)?

Scenario	Static Approach	Impact
333K visits in 1 day	All static files	**No impact** — Cloudflare CDN handles it
10K orders in 1 day	Direct to Apps Script	**Hits 50% of daily limit** — still fine

1M visits in 1 day	All static files	**No impact** — proven to handle billions

The beauty of the static approach: **there is no upper limit on traffic for browsing**. Cloudflare has publicly stated their free plan handled 1.4 billion requests during a DDoS attack without issues. Your 1M visits/month is trivial for their CDN.

Data Freshness Trade-offs

Data Type	Current Freshness	New Freshness	Acceptable?
Product catalog	Real-time (every page load)	Every 4 hours	Yes — products don't change hourly
Stock levels	Real-time	Every 4 hours	Mostly yes — see mitigation below
New images	Instant (Cloudinary)	Next sync (4 hours)	Yes — admin can trigger manual sync
Orders	Real-time	Real-time (unchanged)	Yes — still direct to Apps Script
Prices	Real-time	Every 4 hours	Yes — prices rarely change mid-day

Mitigating Stock Freshness

For a COD (Cash on Delivery) store, slightly stale stock is acceptable because:

1. Orders are confirmed by phone anyway (standard practice in Algeria)
2. If an item sells out between syncs, the admin cancels the order (normal workflow)
3. The admin can trigger a manual GitHub Actions run for immediate sync after big sales

For critical stock updates (flash sales), add a manual trigger button in the admin panel:

TypeScript

```
// Admin panel: "Sync Now" button
const handleForceSync = async () => {
  // Trigger GitHub Actions workflow via repository dispatch
  await fetch('https://api.github.com/repos/OWNER/REPO/dispatches', {
```

```
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${GITHUB_PAT}`,
      'Accept': 'application/vnd.github.v3+json',
    },
    body: JSON.stringify({ event_type: 'force-sync' } ),
  });
  toast.success("Sync triggered — site updates in ~3 minutes");
};
```

Complete Free Tier Budget

Service	What It Does	Monthly Usage	Monthly Limit	Cost
Cloudflare Pages (static)	Serves HTML, JS, CSS, JSON, images	15M requests, ~2TB	Unlimited	**$0**
Cloudflare Pages (builds)	Deploys on git push	180 builds	500 builds	**$0**
GitHub Actions	Syncs data + images every 4h	360 min (private) or unlimited (public)	2,000 min	**$0**
GitHub repo	Stores code + static data	~50 MB	Unlimited (public) / 500 MB (private)	**$0**
Google Apps Script	Orders + admin saves	~700 calls/day	20,000/day	**$0**
Google Sheets	Stores products + orders	~5 MB	10 million cells	**$0**
Cloudinary	Fallback only (new uploads before sync)	~1 GB/month	25 GB/month	**$0**
TOTAL				**$0/month**

Implementation Checklist

Phase 1: Make It Static (1 day)

Plain Text

```
☐ 1. Create scripts/generate-static-data.py (fetches products + stock → static
☐ 2. Run it once manually to generate /public/data/products.json and /public/da
☐ 3. Update use-catalog.ts to fetch from /data/products.json instead of Apps Sc
☐ 4. Update use-stock.ts to fetch from /data/stock.csv instead of Apps Script
☐ 5. Add output: 'export' to next.config.ts (pure static build)
☐ 6. Remove all API routes (src/app/api/) — they're dead code anyway
☐ 7. Test locally: next build && npx serve out/
☐ 8. Verify: products load, images show, checkout works
```

Phase 2: Automate Sync (1 hour)

Plain Text

```
☐ 9. Update .github/workflows/auto-sync.yml (add generate-static-data step, cha
☐ 10. Push to GitHub
☐ 11. Verify GitHub Actions runs successfully
☐ 12. Verify Cloudflare Pages auto-deploys with fresh data
☐ 13. Test the full flow: admin adds product → wait 4h → product appears on sit
☐ 14. Test manual trigger: GitHub Actions → workflow_dispatch → site updates in
```

Phase 3: Optimize (1 day)

Plain Text

```
☐ 15. Remove unused dependencies (next-auth, next-intl, prisma, sharp, recharts
☐ 16. Remove dead code (db.ts, all API routes, health-monitor Workers calls)
☐ 17. Ensure all images are served locally (verify image manifest covers all pr
☐ 18. Add Cache-Control headers in public/_headers for static JSON/CSV:
       /data/products.json → Cache-Control: public, max-age=300
       /data/stock.csv → Cache-Control: public, max-age=300
☐ 19. Test with throttled network (Chrome DevTools → Slow 3G) — should still lo
☐ 20. Final deploy and verify everything works
```

What If You Need Real-Time Stock Later?

If the business grows and real-time stock becomes critical, you can add a single Worker endpoint without exceeding free limits:

Plain Text

```
Visitors/day: 33,333
Stock API calls: 33,333/day (1 per visit)
Workers free limit: 100,000/day
Headroom: 67% free
```

This means you can add real-time stock checking later and still stay 100% free up to ~100,000 visits/day (3M/month).

Comparison: Current vs. Optimized

Metric	Current Architecture	Free Optimized Architecture
Max free traffic	~600 visits/day (Apps Script limit)	**Unlimited** (static)
Page load speed	3-8 seconds (Apps Script fetch)	**<1 second** (static CDN)
Monthly cost	$0	**$0**
Single point of failure	Apps Script (rate limits, downtime)	**None** (static files always available)
Data freshness	Real-time	Every 4 hours (orders still real-time)
Handles viral traffic	No (breaks at 20K visits/day)	**Yes** (Cloudflare handles billions)
Image bandwidth limit	25 GB/month (Cloudinary)	**Unlimited** (Cloudflare Pages)
Requires Workers/Functions	Yes (broken, returns 500)	**No** (pure static)
Build complexity	Next.js 16 + Cloudflare adapter (broken)	Static export (simple, reliable)

Summary

The solution is elegantly simple: **stop treating a catalog website like a real-time application.**

Products don't change every second. Stock doesn't need millisecond accuracy for a COD store. The only thing that needs to be real-time is order submission — and that's only 667 calls/day, well within Apps Script's free limits.

By generating static JSON files every 4 hours and serving everything from Cloudflare's unlimited free CDN, the site can handle 1 million visits, 10 million visits, or even 100 million visits without spending a single cent. The CDN has no cap. The bandwidth has no cap. The only limit is the 500 builds/month — and at 180 builds/month, you're using only 36% of that.

Total monthly cost: $0. Total traffic capacity: effectively unlimited.